


# NPTEL Course on "Computer System Optimization for ML"

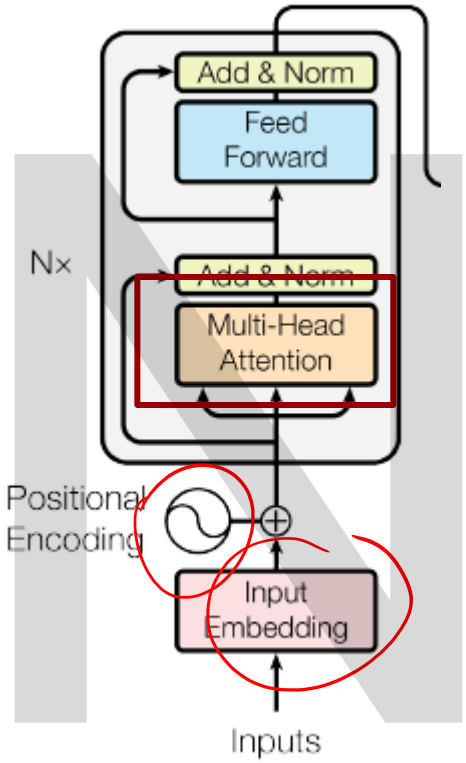
Lecture #25

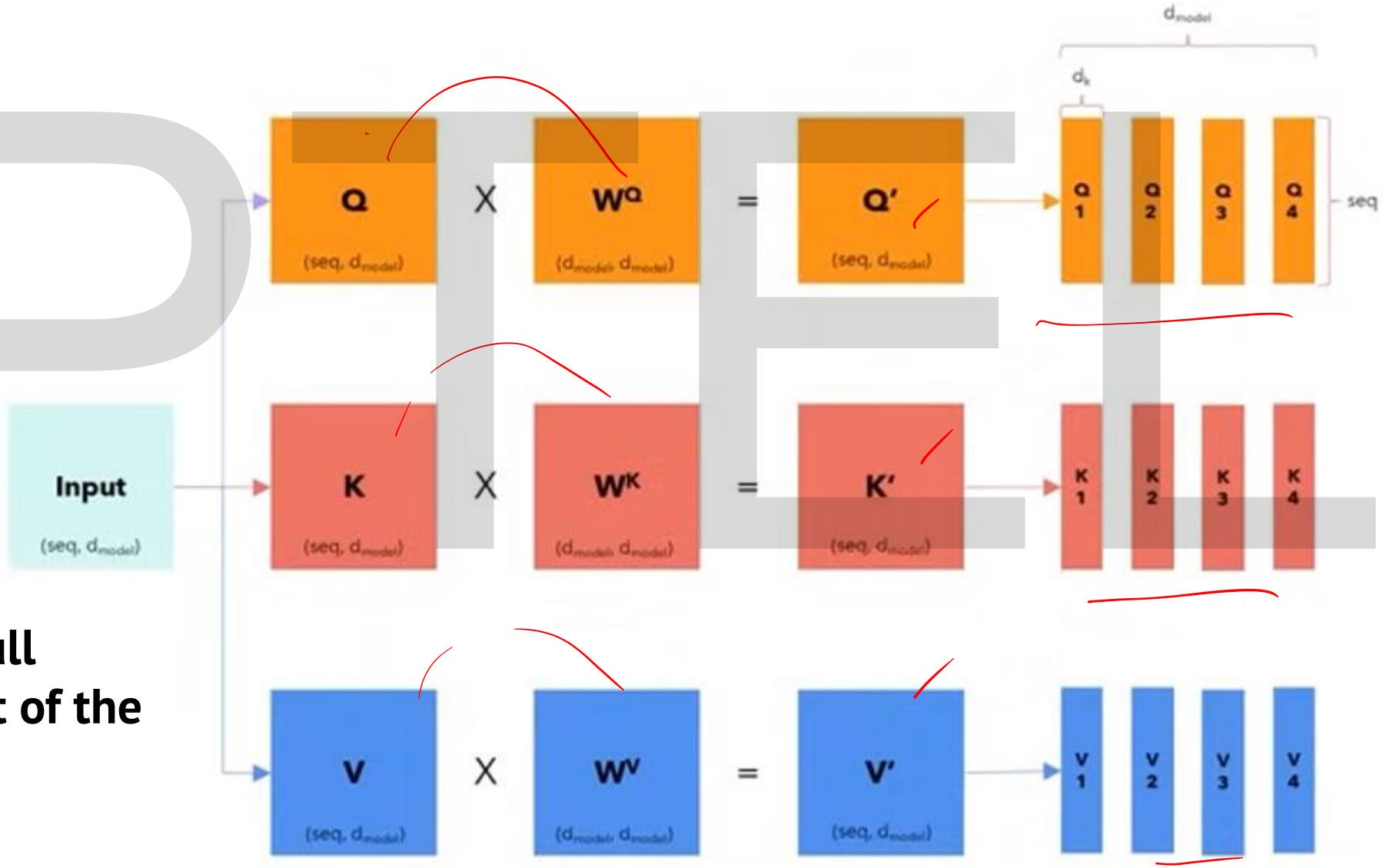
# Agenda

- Recap
- Basics of LLM (contd.)
- Acceleration of LLMs

NPTEL

# Multi-head Attention

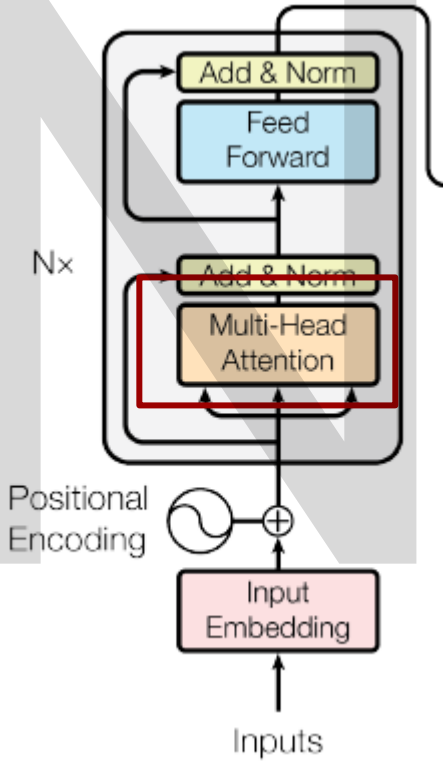


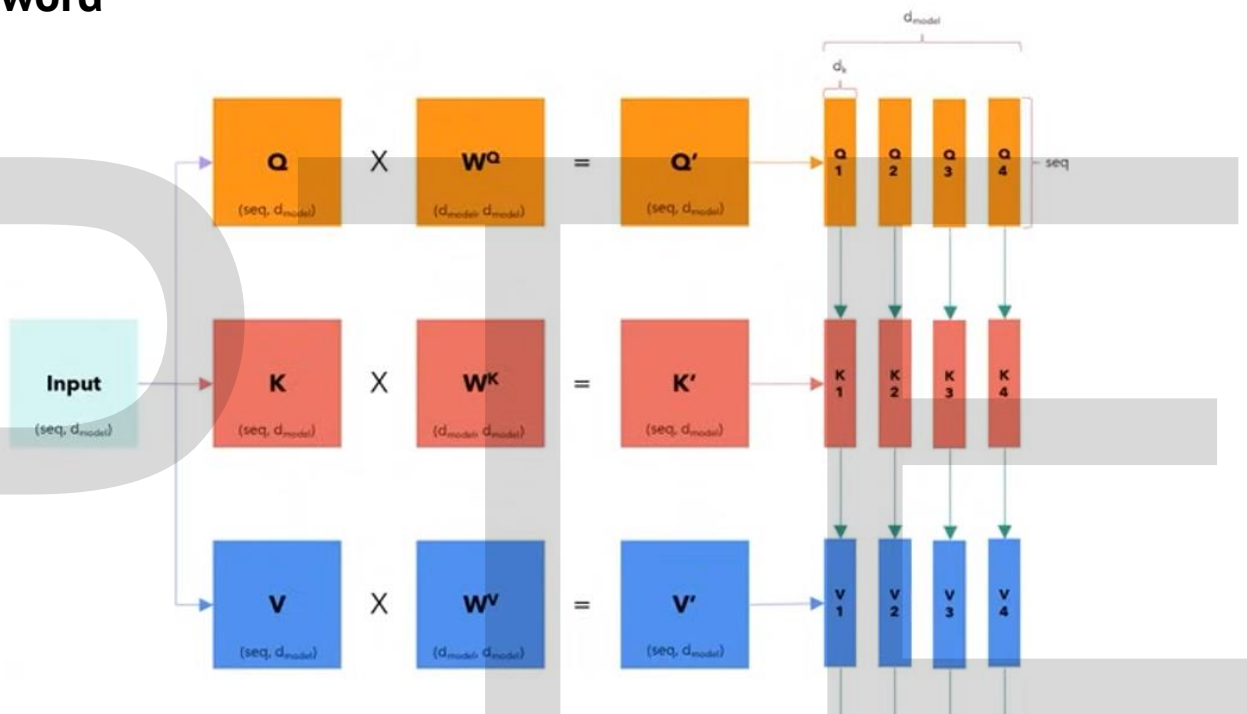




**Every head will see the full sentence but smaller part of the embedding of each word**

# Multi-head Attention

**Each head learns different aspect of a word**




$$Attention(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

$d_k = \frac{d_{model}}{\#Heads}$

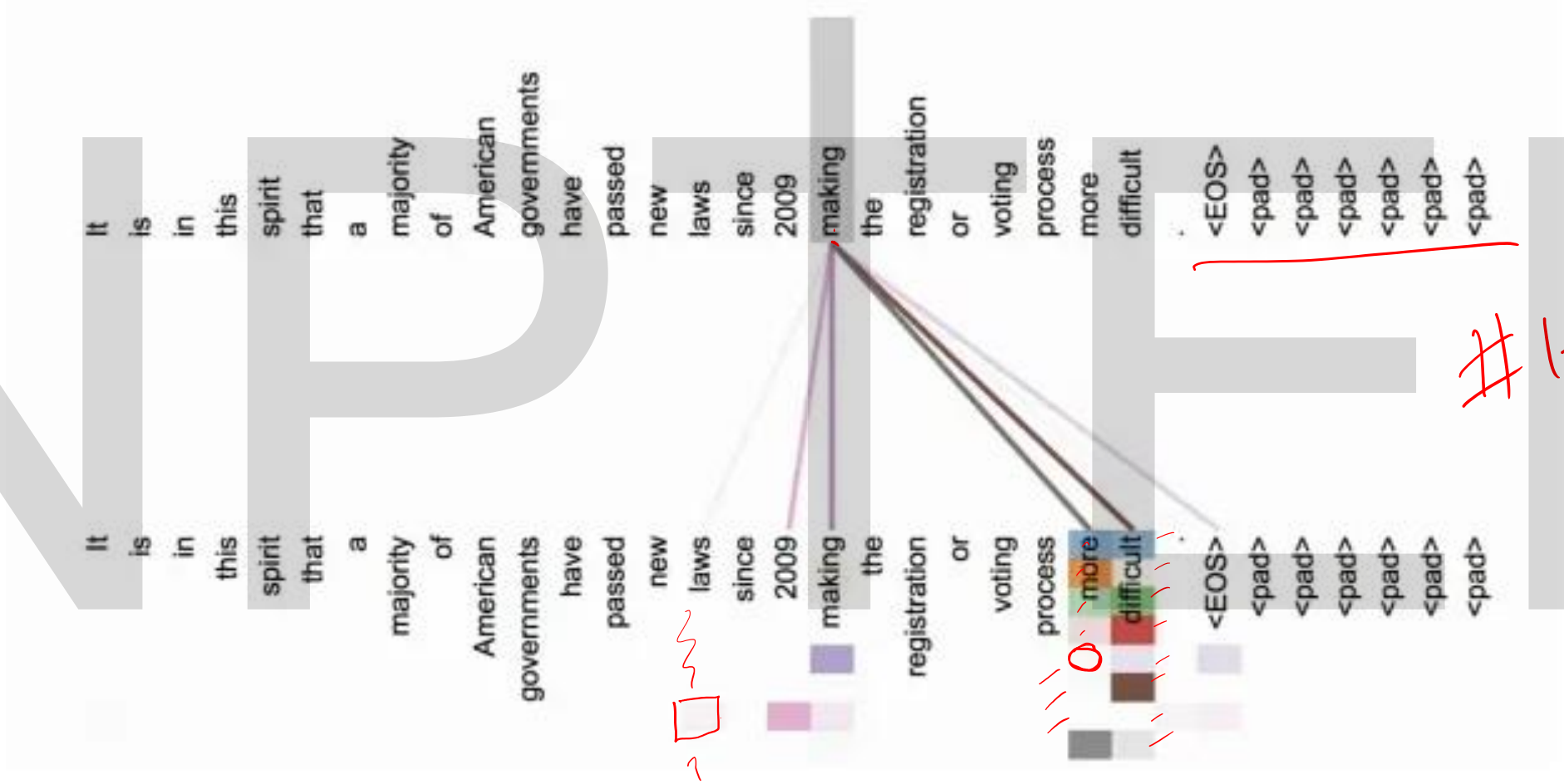
$$MultiHead(Q,K,V) = Concat(head_1 \ldots head_h)W^O$$

# Agenda

- Recap
- **Basics of LLM (contd.)**
- Acceleration of LLMs

NPTEL

# Visualizing Attentions

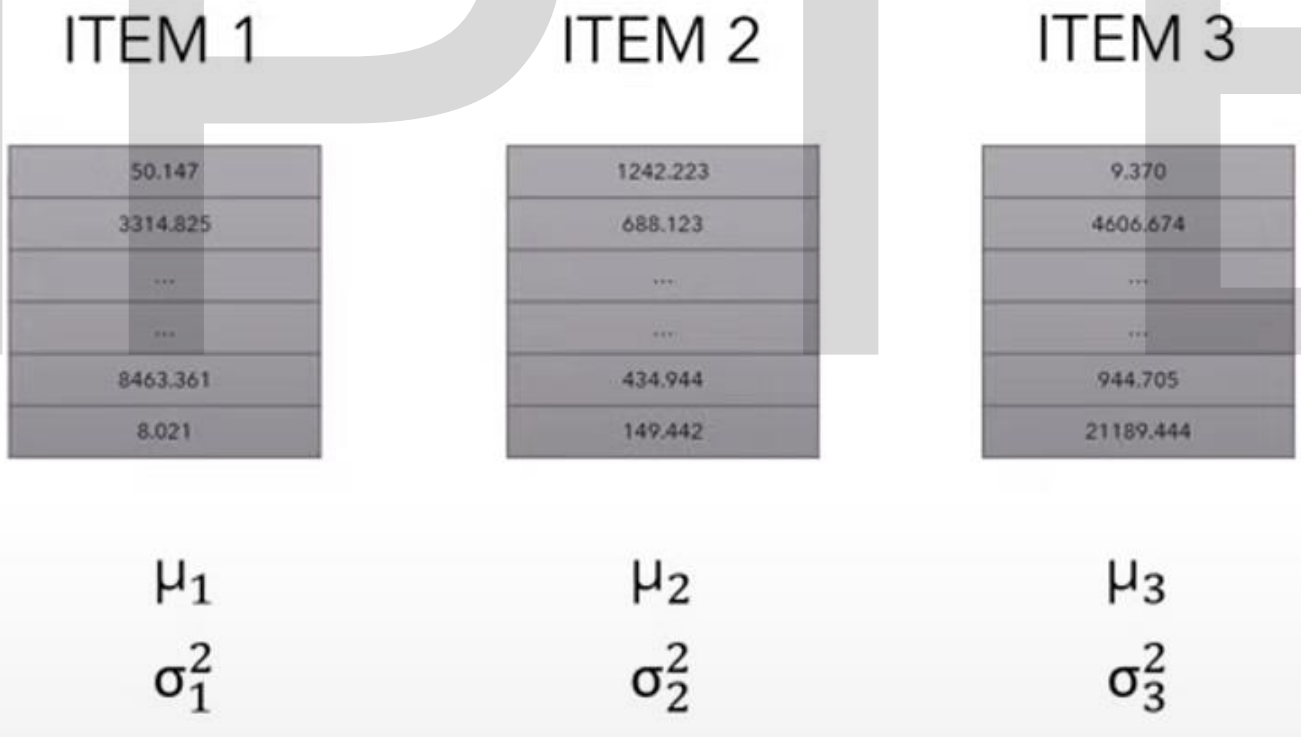


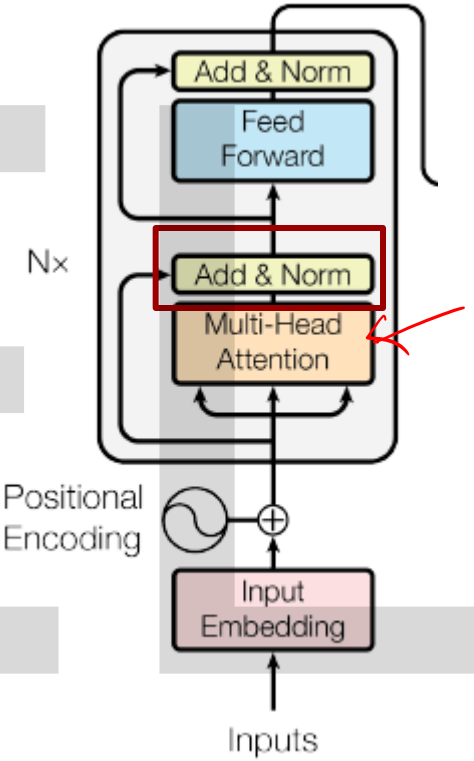

NPTEL

# Encoder: Add and Norm

- **Similar to batch normalization in DNN**

$$-\hat{x}_j = \frac{x_j - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

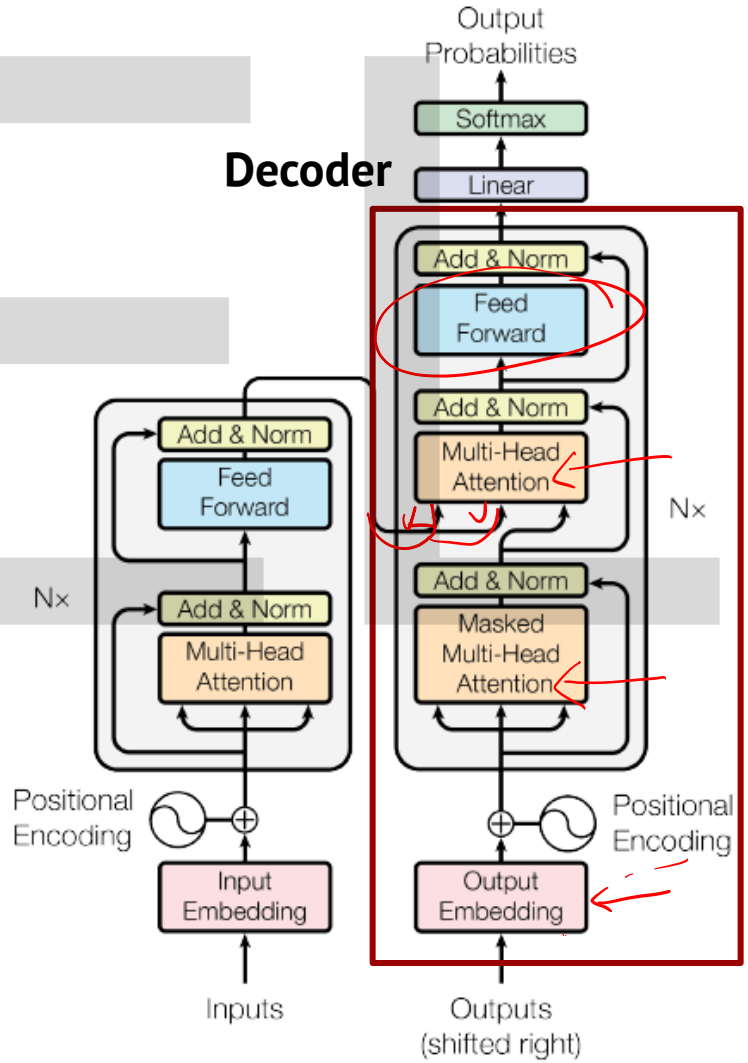



NPTEL

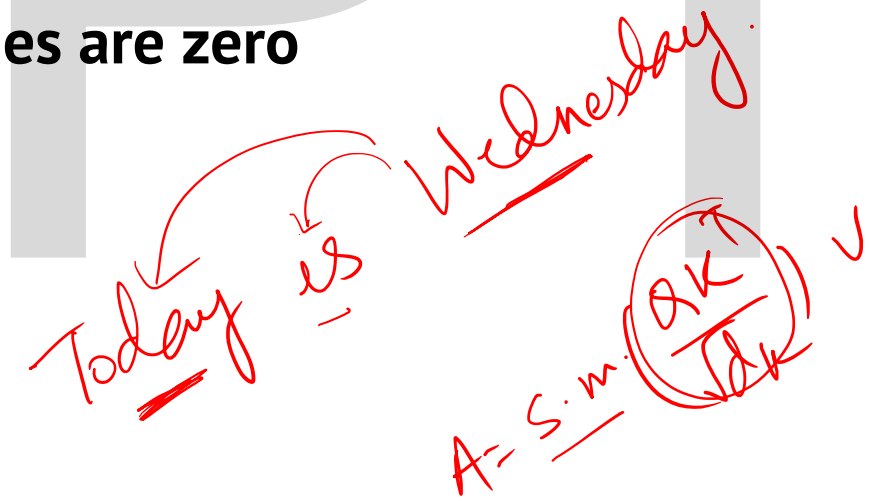
# Decoder

- Output embedding is similar to the input embedding
- Masked Multi-head Attention is same as self-attention on Decoder inputs
- Multi-head Attention
  - Keys (K) and Values (V) are coming from Encoder
  - Queries (Q) are coming from itself
  - No longer self-attention
  - Cross-attention
- Feed forward is a fully connected layer



# Decoder: Masked Multi-Headed Attention

- Goal is to make the model causal
  - Output at a certain position can only depend on the words on the previous positions
  - The model must not see future words
- The values are made $-\infty$
  - Softmax values are zero



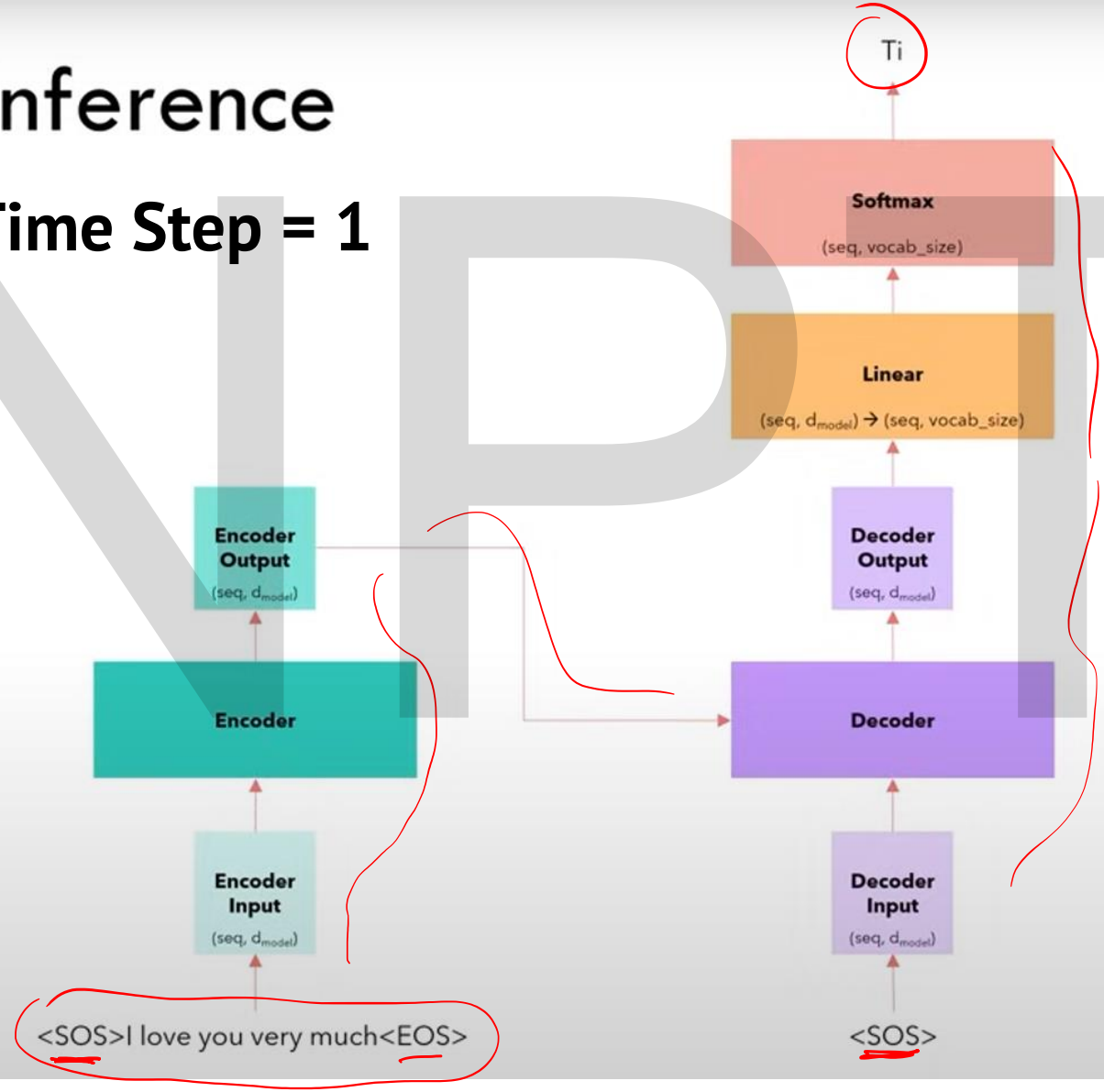

| | YOUR | CAT | IS | A | LOVELY | CAT |
|---|---|---|---|---|---|---|
| YOUR | 0.268 | -∞ | 0.134 | 0.148 | 0.179 | 0.152 |
| CAT | 0.124 | 0.278 | 0.201 | 0.128 | 0.154 | 0.115 |
| IS | 0.147 | 0.132 | 0.262 | 0.097 | 0.218 | 0.145 |
| A | 0.210 | 0.128 | 0.206 | 0.212 | 0.119 | 0.125 |
| LOVELY | 0.146 | 0.158 | 0.152 | 0.143 | 0.227 | 0.174 |
| CAT | 0.195 | 0.114 | 0.203 | 0.103 | 0.157 | 0.229 |

NPTEL

# Inference

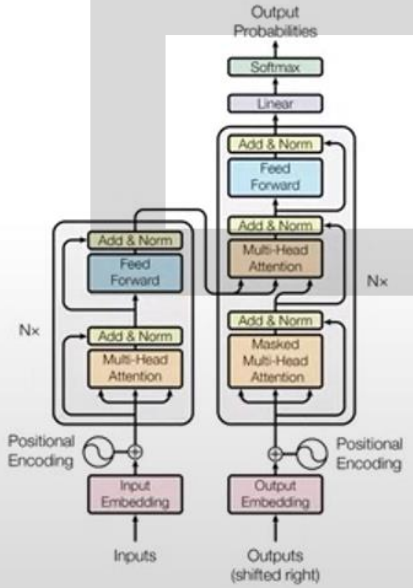
## Inference

### Time Step = 1




We select a token from the vocabulary corresponding to the position of the token with the maximum value.

The output of the last layer is commonly known as **logits**




* Both sequences will have same length thanks to padding

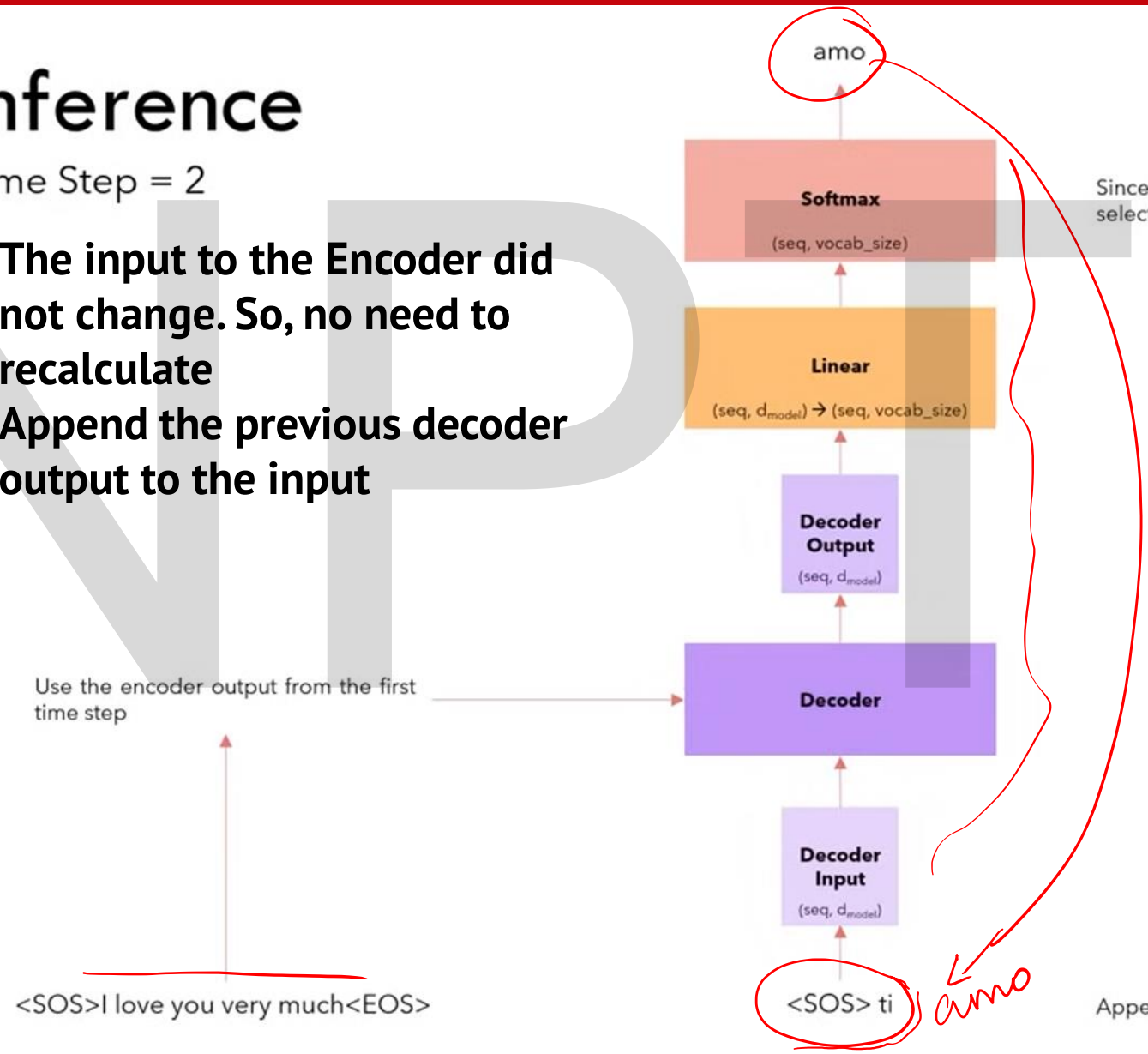
NPTEL

# Inference

## Inference

Time Step = 2

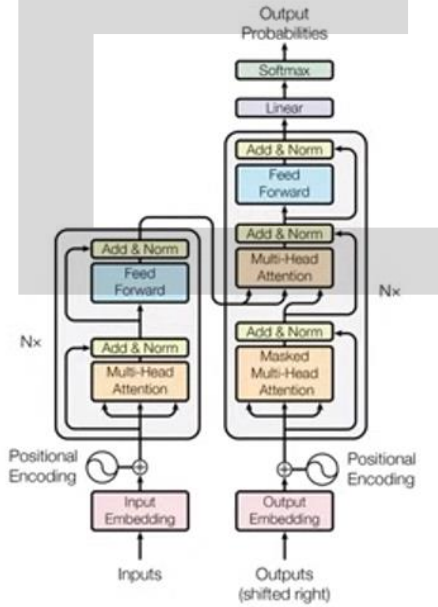
- **The input to the Encoder did not change. So, no need to recalculate**
- **Append the previous decoder output to the input**




Use the encoder output from the first time step

<SOS>I love you very much<EOS>

Since decoder input now contains **two** tokens, we select the softmax corresponding to the second token.

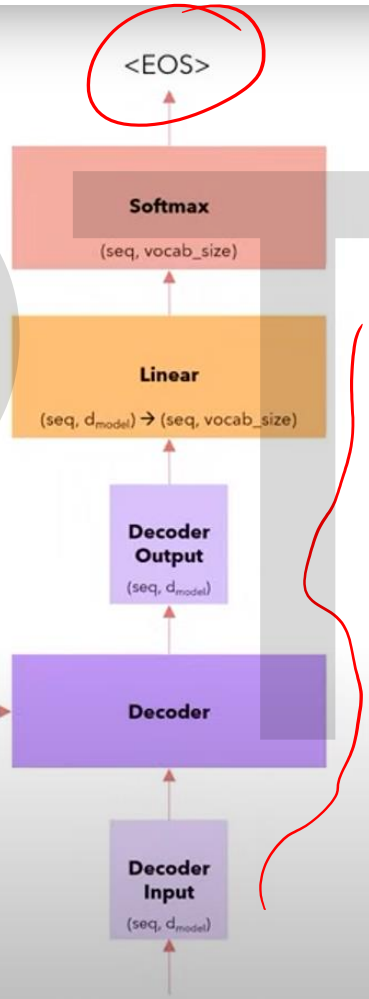



Append the previously output word to the decoder input

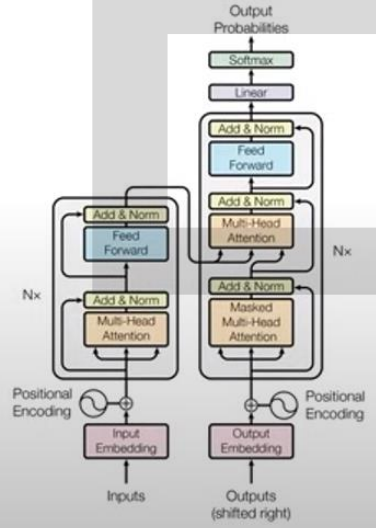
# Inference

## Inference

Time Step = 4




Since decoder input now contains **four** tokens, we select the softmax corresponding to the fourth token.




Use the encoder output from the first time step

<SOS>I love you very much<EOS>

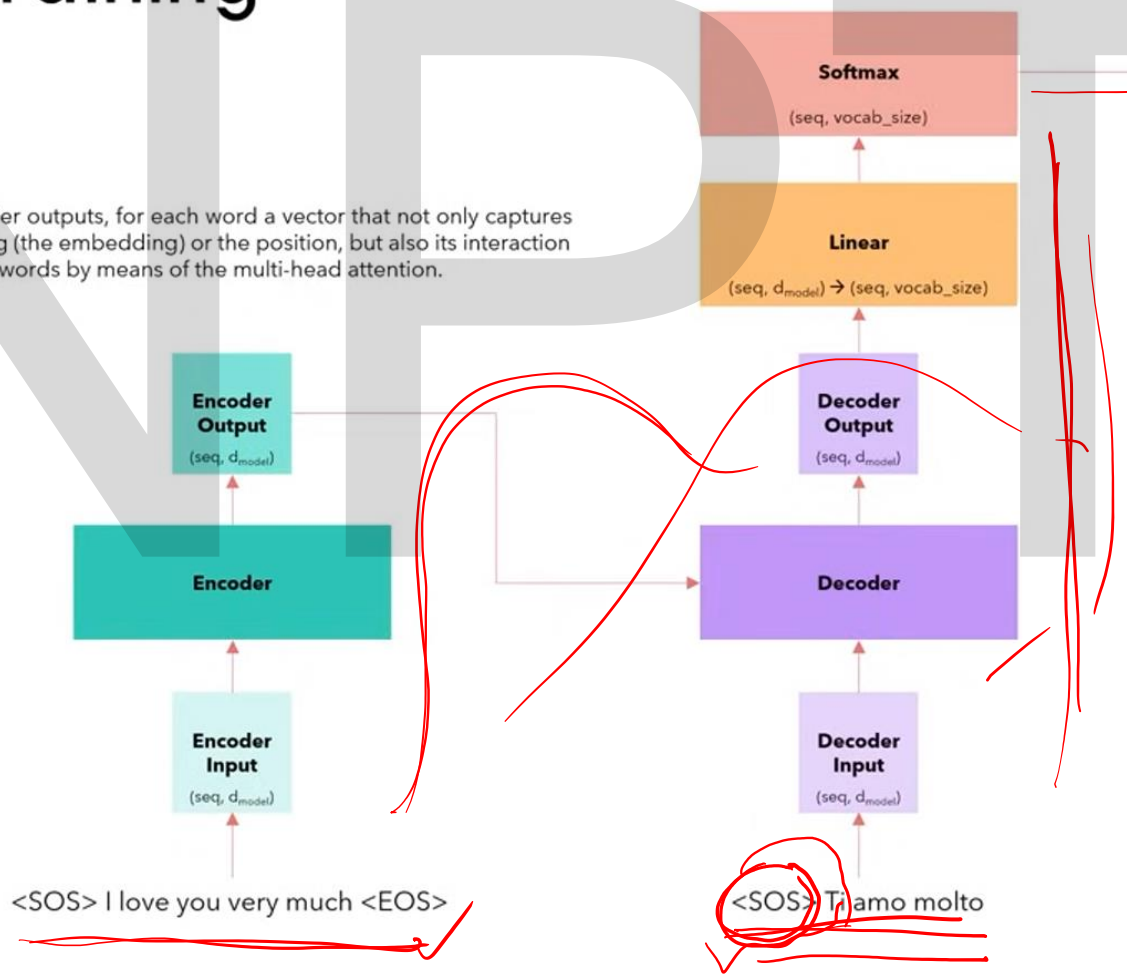
<SOS> ti amo molto

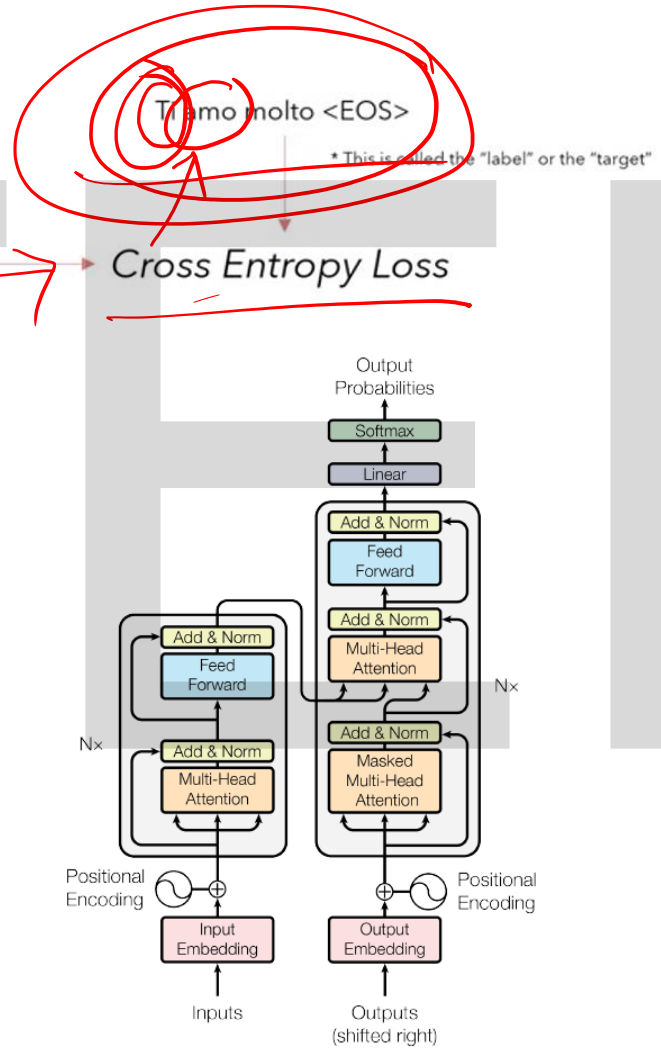
Append the previously output word to the decoder input

# Training of a Transformer Model

## Training

The encoder outputs, for each word a vector that not only captures its meaning (the embedding) or the position, but also its interaction with other words by means of the multi-head attention.

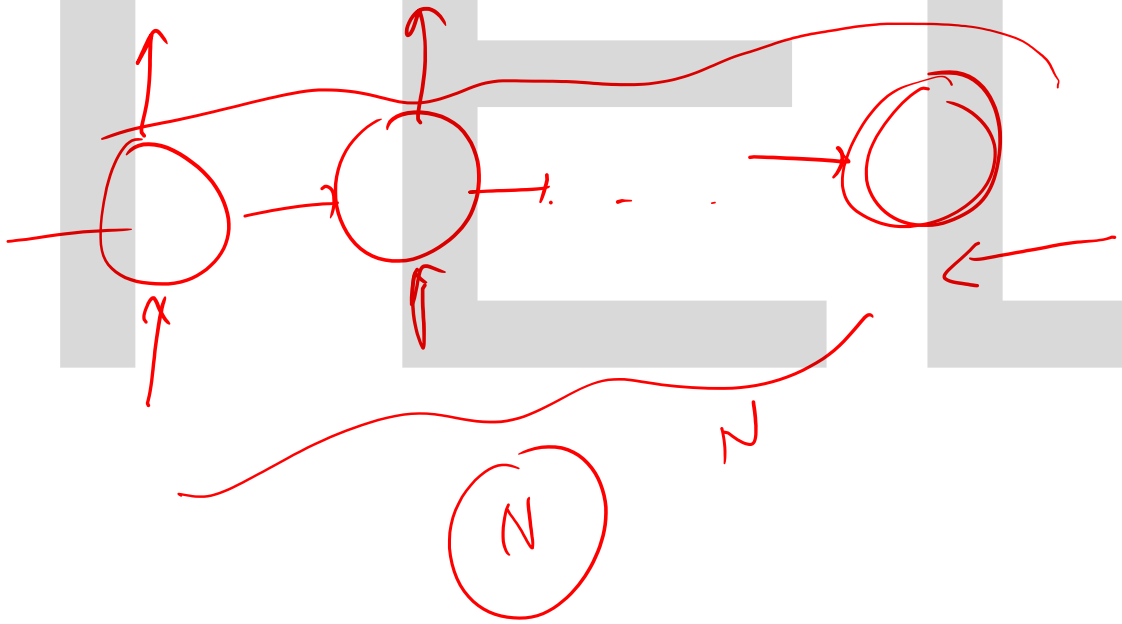



We prepend the <SOS> token at the beginning. That's why the paper says that the decoder input is shifted right.

# Training of a Transformer Model

- <SOS> Ti amo molto
- Ti amo molto <EOS>
- It all happens in one time step
  - Unlike RNNs




NPTEL

# Agenda

- Recap
- Basics of LLM (contd.)
- **Acceleration of LLMs**

NPTEL

# LinFormer (Reducing the Complexity to O(n))

- $head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$
- $= softmax\left(\frac{QW_i^Q(KW_i^K)^T}{\sqrt{d_k}}\right)VW_i^V$
- Computing the inner matrix ($P$) is expensive
  - $O(n^2)$ complexity, where $N$ is the sequence length
- Observation: Self-attention is low rank
  - Many rows of $P$ are almost dependent to each other
  - Singular Value Decomposition (SVD) can be done

NPTEL

# LinFormer (Reducing the Complexity to O(n))

- $head_i = Attention(QW_i^Q, E_i KW_i^K, F_i VW_i^V)$
  - $E_i$ and $F_i$ are $n \times k$ projection matrices where $k \ll n$
- $softmax\left(\frac{QW_i^Q (E_i KW_i^K)^T}{\sqrt{d_k}}\right) F_i VW_i^V$
- The dimension of the inner matrix is $n \times k$
- Computational complexity $O(nk) \cong O(n)$
  - $k = O\left(\frac{d}{\epsilon^2}\right)$
  - $d$ is the embedding size
  - Independent of n

NPTEL

# Results: Accuracy

| $n$ | Model | SST-2 | IMDB | QNLI | QQP | Average |
|---|---|---|---|---|---|---|
| 512 | Liu et al. (2019), RoBERTa-base | 93.1 | 94.1 | 90.9 | **90.9** | 92.25 |
| | Linformer, 128 | 92.4 | 94.0 | 90.4 | 90.2 | 91.75 |
| | Linformer, 128, shared kv | **93.4** | 93.4 | 90.3 | 90.3 | 91.85 |
| | Linformer, 128, shared kv, layer | 93.2 | 93.8 | 90.1 | 90.2 | 91.83 |
| | Linformer, 256 | 93.2 | 94.0 | 90.6 | 90.5 | 92.08 |
| | Linformer, 256, shared kv | 93.3 | 93.6 | 90.6 | 90.6 | 92.03 |
| | Linformer, 256, shared kv, layer | 93.1 | 94.1 | **91.2** | 90.8 | **92.30** |
| 512 | Devlin et al. (2019), BERT-base | 92.7 | 93.5 | 91.8 | 89.6 | 91.90 |
| | Sanh et al. (2019), Distilled BERT | 91.3 | 92.8 | 89.2 | 88.5 | 90.45 |
| 1024 | Linformer, 256 | 93.0 | 93.8 | 90.4 | 90.4 | 91.90 |
| | Linformer, 256, shared kv | 93.0 | 93.6 | 90.3 | 90.4 | 91.83 |
| | Linformer, 256, shared kv, layer | 93.2 | **94.2** | 90.8 | 90.5 | 92.18 |

# Results: Improvement in Efficiency

**Time Saved**

| length $n$ | projected dimensions $k$ | | | | |
|---|---|---|---|---|---|
| | 128 | 256 | 512 | 1024 | 2048 |
| 512 | 1.5x | 1.3x | - | - | - |
| 1024 | 1.7x | 1.6x | 1.3x | - | - |
| 2048 | 2.6x | 2.4x | 2.1x | 1.3x | - |
| 4096 | 3.4x | 3.2x | 2.8x | 2.2x | 1.3x |
| 8192 | 5.5x | 5.0x | 4.4x | 3.5x | 2.1x |
| 16384 | 8.6x | 7.8x | 7.0x | 5.6x | 3.3x |
| 32768 | 13x | 12x | 11x | 8.8x | 5.0x |
| 65536 | 20x | 18x | 16x | 14x | 7.9x |

**Memory Saved**

| length $n$ | projected dimensions $k$ | | | | |
|---|---|---|---|---|---|
| | 128 | 256 | 512 | 1024 | 2048 |
| 512 | 1.7x | 1.5x | - | - | - |
| 1024 | 3.0x | 2.9x | 1.8x | - | - |
| 2048 | 6.1x | 5.6x | 3.6x | 2.0x | - |
| 4096 | 14x | 13x | 8.3x | 4.3x | 2.3x |
| 8192 | 28x | 26x | 17x | 8.5x | 4.5x |
| 16384 | 56x | 48x | 32x | 16x | 8x |
| 32768 | 56x | 48x | 36x | 18x | 16x |
| 65536 | 60x | 52x | 40x | 20x | 18x |




# THANK YOU

Slide Courtesy: Murali Annavaram (University of Southern California), R. Govindarajan (IISc), Arkaprava Basu (IISc), Jayashree Mohan (Microsoft Research)